MANUAL OPERATIVO

Sistema de Pronóstico Numérico Atmosférico

`WRF-Chem — run_pronostico.sh`

Versión 3.0	**Institución** ICAyCC — UNAM
Autor J. A. García Reynoso	**Contacto** agustin@atmosfera.unam.mx

1. Introducción

Este manual describe la operación del sistema de pronóstico numérico de calidad del aire de la Zona Metropolitana del Valle de México (ZMVM), basado en el modelo WRF-Chem. El sistema está diseñado para ejecutarse automáticamente cada día vía cron en un servidor de 8 cores, sin gestor de colas tipo SLURM.

El script principal run_pronostico.sh orquesta la descarga de condiciones iniciales del modelo global GFS (NOAA), el pre-procesamiento meteorológico con WPS, la inicialización y la simulación de 72 horas con WRF-Chem.

Objetivo del sistema

Generar pronósticos de 72 horas de variables meteorológicas y de calidad del aire para la ZMVM con una resolución horizontal de 3 km, en ciclo operativo 00Z diario.

2. Arquitectura del Flujo de Trabajo

El pipeline se divide en 6 pasos secuenciales, con la etapa de cálculo de emisiones ejecutándose en paralelo (background) mientras se procesan los datos meteorológicos.

Paso	Descripción
PASO 1 (BG)	Cálculo de inventario de emisiones (ecacor.sh) — se lanza en background
PASO 2	Carga de módulos WPS y descarga de datos GFS (S3/NOAA) con reintentos automáticos
PASO 3	Pre-procesamiento WPS: link_grib → ungrib.exe → metgrid.exe
PASO 4	Carga de módulos WRF y ejecución de real.exe (4 MPI)
PASO 5	Sincronización con emisiones y ejecución de wrf.exe (8 MPI)
PASO 6	Análisis de supervisión post-proceso (analiza2.sh)

2.1 Diagrama de dependencias

```
[PASO 1] ecacor.sh ──────────────────────────────────────────┐
                                                             |
[PASO 2] Descarga GFS (con reintentos automáticos)           |
     |  Intento 1 → fallo → espera 5 min                     |
     |  Intento 2 → fallo → espera 5 min                     |
     |  Intento 3 → fallo → ABORTAR   → éxito → continúa     |
```

```
    ▼                                                          │
[PASO 3] WPS: link_grib → ungrib → metgrid                    │
    ▼                                                          │
[PASO 4] real.exe (4 MPI)                                      │
    ▼                        wait ◄──────────────────────────┘
[PASO 5] wrf.exe (8 MPI)
    ▼
[PASO 6] analiza2.sh
```

3. Requisitos de Software

Componente	Versión / Notas
Bash	≥ 4.4 (requerido por set -euo pipefail)
AWS CLI	v2 — descarga de datos GFS desde NOAA S3 público
OpenMPI / MPICH	Compatible con el WRF compilado en el sistema
Spack	Gestor de entornos de software científico HPC
WPS 4.4	Pre-procesamiento WRF (link_grib, ungrib, metgrid)
WRF-Chem	Modelo de pronóstico (real.exe, wrf.exe)
NCO (ncks)	Para condiciones iniciales de química del día anterior
bc, awk, date	GNU Coreutils — cálculo de métricas y fechas

3.1 Requisitos de hardware

Recurso	Mínimo recomendado
CPUs	8 cores (configuración operativa actual)
RAM	32 GB (WRF-Chem requiere ≥ 16 GB)
Almacenamiento	100 GB libres en WORK_DIR
Red	Acceso a internet para S3 (s3://noaa-gfs-bdp-pds)

4. Preparación del Entorno

4.1 Estructura de directorios

El script crea automáticamente los directorios si no existen. La estructura esperada es:

```
/shared/pronostico/
├── run_pronostico.sh       ← script maestro
├── analiza2.sh             ← análisis post-proceso
├── emis_2016/
│   └── ecacor.sh           ← cálculo de emisiones
├── data/                   ← datos GFS descargados
├── wpsprd/                 ← directorio de trabajo WPS
│   ├── ungrib.exe
│   ├── metgrid.exe
│   └── link_grib.csh
├── wrfprd/                 ← directorio de trabajo WRF
│   ├── real.exe
│   └── wrf.exe
├── salidas/                ← archivos wrfout_d01_*
└── registro_eventos/
    ├── performance_YYYY-MM-DD.log
    └── emisiones_YYYY-MM-DD.log
```

4.2 Verificación previa a la ejecución

Antes de ejecutar el pipeline, verificar:

1. Acceso a S3 de NOAA (requiere salida a internet):

```
aws s3 ls s3://noaa-gfs-bdp-pds/gfs.$(date +%Y%m%d)/00/atmos/ \
    --no-sign-request | head -5
```

2. Disponibilidad de módulos Spack:

```
spack load wps && which ungrib.exe
spack load /hons4ds && which wrf.exe
```

3. Permisos de ejecución:

```
chmod +x /shared/pronostico/run_pronostico.sh
```

5. Ejecución Paso a Paso

5.1 Ejecución manual

```
cd /shared/pronostico
./run_pronostico.sh
```

5.2 Ejecución con monitoreo en tiempo real

```
./run_pronostico.sh &
tail -f registro_eventos/performance_$(date +%Y-%m-%d).log
```

5.3 Configuración de cron (ejecución automática)

Para ejecución automática diaria a las 06:00 UTC:

```
crontab -e
# Agregar la siguiente línea:
0 6 * * * /shared/pronostico/run_pronostico.sh \
    >> /shared/pronostico/registro_eventos/cron.log 2>&1
```

Nota sobre el horario cron
El ciclo 00Z del GFS suele estar disponible entre las 05:00 y 06:00 UTC.
Si se configura cron a las 06:00 UTC y los datos no están disponibles,
el sistema de reintentos esperará hasta 15 minutos adicionales (3 intentos × 5 min).
Ajustar el horario o los parámetros de reintento según la latencia observada.

6. Sistema de Reintentos para Descarga GFS

Los datos GFS se publican en S3 con un retraso variable respecto al tiempo nominal del ciclo. El script implementa un mecanismo de reintentos automático para manejar esta indisponibilidad temporal.

6.1 Lógica de reintentos

```
Para cada archivo gfs.t00z.pgrb2.0p25.fHHH:


  Intento 1
    ├── Éxito → archivo validado (tamaño > 0) → continúa
    └── Fallo → registra WARNING → espera 300s (5 min)
          ▼
  Intento 2
    ├── Éxito → continúa
    └── Fallo → registra WARNING → espera 300s
          ▼
  Intento 3 (final)
    ├── Éxito → continúa
    └── Fallo → registra ERROR → ABORTAR pipeline
```

6.2 Parámetros configurables

Ubicados en la Sección 1 del script (variables readonly):

Variable	Default	Descripción
GFS_MAX_RETRIES	3	Número máximo de intentos totales por archivo
GFS_RETRY_WAIT	300	Segundos de espera entre reintentos (300 = 5 min)

6.3 Modificar los parámetros de reintento

```
# Editar en la Sección 1 del script:
readonly GFS_MAX_RETRIES=5    # Aumentar a 5 intentos
readonly GFS_RETRY_WAIT=600   # Aumentar a 10 minutos entre intentos
```

Fallo definitivo de descarga GFS

Si se agotan todos los intentos, el log registrará:

```
[ERROR] FALLO DEFINITIVO EN DESCARGA GFS
[ERROR] Los datos GFS NO estaban disponibles en el servidor.
[ERROR] PIPELINE ABORTADO por falta de datos GFS.
```

El script termina con código de salida 1. No se generará pronóstico para ese día.
Acción: verificar manualmente la disponibilidad en s3://noaa-gfs-bdp-pds/

7. Interpretación de Logs

El archivo de log principal es: registro_eventos/performance_YYYY-MM-DD.log

7.1 Niveles de severidad

Nivel	Significado
[INFO]	Operación iniciada o información de estado normal
[OK]	Etapa completada exitosamente
[WARNING]	Situación anormal no crítica (ej. reintento de descarga)
[ERROR]	Fallo crítico que ha causado la interrupción del pipeline
[METRIC]	Métrica de rendimiento (tiempo, memoria, CPU)

7.2 Ejemplo de log con reintento exitoso

```
2025-07-16 06:05:01 [INFO]    - Intento 1/3 para: gfs.t00z.pgrb2.0p25.f000

2025-07-16 06:05:02 [WARNING] - ✗ Intento 1/3 fallido: gfs.t00z.pgrb2.0p25.f000

2025-07-16 06:05:02 [WARNING] - ⏳ Esperando 5 minuto(s) (300s) antes del
reintento...

2025-07-16 06:10:03 [INFO]    - Intento 2/3 para: gfs.t00z.pgrb2.0p25.f000

2025-07-16 06:10:45 [OK]      - ✓ Archivo obtenido exitosamente (intento 2):
gfs.t00z.pgrb2.0p25.f000
```

7.3 Ejemplo de log de métricas WRF

```
2025-07-16 14:32:10 [METRIC] - Tiempo de ejecución (wall clock): 7:46:53

2025-07-16 14:32:10 [METRIC] - Tiempo de usuario: 221813.45 s

2025-07-16 14:32:10 [METRIC] - Uso de CPU: 793%

2025-07-16 14:32:10 [METRIC] - Memoria máxima: 18432 MB (18874368 kB)
```

8. Manejo de Errores

8.1 Comportamiento ante fallos

El script usa set -euo pipefail, lo que significa que cualquier comando que falle detendrá inmediatamente la ejecución. La función run_and_check() envuelve cada comando crítico y registra el error en el log antes de abortar.

8.2 Errores comunes y soluciones

Error en Log	Causa probable	Acción correctiva
FALLO DEFINITIVO EN DESCARGA GFS	Datos no disponibles en NOAA S3	Verificar disponibilidad; aumentar GFS_RETRY_WAIT
ungrib.exe falló	Archivos GRIB incompletos o corruptos	Revisar archivos en data/; re-ejecutar descarga
real.exe falló	met_em corruptos o namelist incorrecto	Revisar WPS; verificar fechas en namelist.wps
wrf.exe falló	Error numérico o namelist inconsistente	Revisar rsl.error.0000 en wrfprd/
Variable no definida (set -u)	Bug en el script o env incompleto	Verificar carga de módulos Spack

8.3 Diagnóstico de fallo de WRF

Si wrf.exe falla, los archivos de diagnóstico más relevantes son:

```
# Revisar el log de error del proceso MPI rank 0:
cat /shared/pronostico/wrfprd/rsl.error.0000


# Ver el log principal del pipeline:
tail -100 /shared/pronostico/registro_eventos/performance_$(date +%Y-%m-%d).log


# Verificar que wrfinput exista y no esté vacío:
ls -lh /shared/pronostico/wrfprd/wrfinput_d01
```

9. Solución de Problemas Comunes

Problema: Los datos GFS nunca están disponibles en el horario programado

Diagnóstico: Verificar la latencia típica de publicación del ciclo 00Z del GFS en NOAA S3.

```
# Verificar disponibilidad manual:
aws s3 ls s3://noaa-gfs-bdp-pds/gfs.$(date +%Y%m%d)/00/atmos/ \
    --no-sign-request | head -3

# Si el directorio no existe, el ciclo aún no se ha publicado.
```

Solución: Retrasar el cron a las 07:00 UTC o aumentar GFS_RETRY_WAIT y GFS_MAX_RETRIES.

Problema: El proceso de emisiones falla pero WRF continúa

El script registra una advertencia [WARNING] si ecacor.sh termina con error, pero no aborta el pipeline por defecto. Si las emisiones son obligatorias, cambiar el comportamiento en la Sección 8 del script para agregar exit 1 al bloque de verificación de $EMISS_PID.

Problema: WRF termina con inestabilidad numérica

Síntomas: rsl.error.0000 contiene "SIGSEGV" o "NaN". Acciones:

4. Reducir el paso de tiempo (time_step) en namelist.input
5. Verificar que los datos de emisiones no contengan valores negativos
6. Revisar la consistencia entre las fechas de start/end en namelist.input y namelist.wps

Problema: Directorio de salidas lleno (disco)

El script elimina automáticamente las salidas con 7 días de antigüedad. Si se necesita mayor retención, modificar la variable de retención en la función run_wrf(). Para limpieza manual:

```
# Listar archivos de salida por tamaño:
du -sh /shared/pronostico/salidas/wrfout_d01_*

# Eliminar salidas de más de 14 días:
find /shared/pronostico/salidas/ -name "wrfout_d01_*" -mtime +14 -delete
```

10. Referencia de Parámetros Configurables

Todos los parámetros se encuentran en la Sección 1 del script. Son variables readonly para prevenir modificaciones accidentales durante la ejecución.

Variable	Default	Descripción
WORK_DIR	/shared/pronostico	Directorio raíz del sistema
FORECAST_HOURS	72	Horizonte de pronóstico en horas
GFS_INTERVAL_HOURS	3	Intervalo entre archivos GFS (horas)
GFS_MAX_RETRIES	3	Máximo de intentos de descarga por archivo
GFS_RETRY_WAIT	300	Segundos entre reintentos GFS
WRF_NPROCS	8	Procesadores MPI para wrf.exe
REAL_NPROCS	4	Procesadores MPI para real.exe

Contacto y soporte técnico

Instituto de Ciencias de la Atmósfera y Cambio Climático — UNAM
Autor: J. A. García Reynoso
Correo: agustin@atmosfera.unam.mx
Versión del manual: 3.0 | Script: run_pronostico.sh v3.0